

Sumário

Especificação da Etapa 3 — Compilador Completo: TAC e Assembly	1
1. Objetivo	1
2. Ponto de Partida	2
3. Técnica de Back-Patching	2
3.1 Funções fornecidas em <code>backpatch.h</code>	3
3.2 Resultado de tradução booleana	3
4. Infraestrutura de Back-Patching (TODOs E3-E e E3-F)	3
TODO-E3-E: <code>patch_list_backpatch(list, label)</code>	3
TODO-E3-F: <code>patch_list_merge(l1, l2)</code>	3
5. Algoritmos de Tradução TAC (TODOs E3-A a E3-D)	4
5.1 TODO-E3-A: Expressões booleanas com curto-circuito	4
5.2 TODO-E3-B: Comando <code>if/else</code>	4
5.3 TODO-E3-C: Comando <code>while...do</code>	5
5.4 TODO-E3-D: Comando <code>for</code>	5
6. Geração de Assembly (Fornecida)	5
6.1 Convenções de chamada (System V x86-64)	5
6.2 Prólogo/epílogo de função	5
6.3 Mapeamento TAC → assembly (exemplos)	6
7. Como compilar e usar	6
8. Testes	6
9. Critérios de Avaliação	7
Apêndice A — Otimizações Locais (Opcional, não avaliado)	7
A.1 Blocos Básicos	7
A.2 Grafo de Fluxo de Controle (CFG)	8
A.3 TODO-OPT-A: <code>cfg_build()</code>	8
A.4 TODO-OPT-B: <code>bb_const_fold()</code>	9
A.5 TODO-OPT-C: <code>bb_copy_prop()</code>	9
A.6 Como usar o módulo	10

Especificação da Etapa 3 — Compilador Completo: TAC e Assembly

INF01083 — Linguagens de Programação II / Compiladores — 2026/1 UFRGS / INF
— Prof. Nicolas Maillard Lab: segunda-feira, 01 de junho de 2026

1. Objetivo

Esta é a **etapa final** do projeto de compilador LC—. O objetivo é completar o pipeline de compilação, produzindo **binários executáveis** a partir de programas LC— escritos com controle de fluxo completo:

programa LC— → AST → TAC completo → assembly x86-64 → binário

Ao final, o compilador deve ser capaz de compilar e executar corretamente programas com expressões aritméticas, `if/else`, `while`, `for`, funções recursivas e arrays.

O que os alunos devem implementar:

- Geração de TAC para expressões booleanas com **curto-circuito** (`&&`, `||`)
- Geração de TAC para controle de fluxo: `if/else`, `while...do`, `for`

O que é fornecido completo:

- Todo o frontend da Etapa 1 (scanner, parser, AST, tabela de símbolos)
- A infraestrutura TAC da Etapa 2 (expressões aritméticas, atribuições, declarações, funções — já funcional)
- O módulo de **back-patching** (`backpatch.h/c`)
- O **gerador de assembly** (`asmgen.h/c`) — traduz automaticamente o TAC para código x86-64 executável; os alunos **não modificam** este arquivo

2. Ponto de Partida

Esta etapa parte da **solução completa da Etapa 2**. O diretório `src/` contém:

Arquivo	Status	O que é
<code>scanner.l</code>	Completo (fornecido)	Analisador léxico
<code>parser.y</code>	Completo (fornecido)	Analisador sintático (bison)
<code>ast.h / ast.c</code>	Completo (fornecido)	Árvore Sintática Abstrata
<code>syntab.h / syntab.c</code>	Completo (fornecido)	Tabela de símbolos
<code>tac.h / tac.c</code>	Completo (fornecido)	Representação e impressão do TAC
<code>backpatch.h / backpatch.c</code>	Completo (fornecido)	Back-patching para desvios condicionais
<code>asmgen.h / asmggen.c</code>	Completo (fornecido)	Gerador de assembly x86-64 — não modificar
<code>codegen.h</code>	Completo (fornecido)	Interface do gerador de TAC
<code>codegen.c</code>	Esqueleto (alunos)	Gerador de TAC — contém os TODOs
<code>main.c</code>	Completo (fornecido)	Pipeline completo: <code>parse</code> → <code>TAC</code> → <code>assembly</code>

3. Técnica de Back-Patching

Expressões booleanas com curto-circuito (`&&`, `||`) requerem emitir saltos condicionais **antes** de saber os rótulos de destino. A técnica de **back-patching** resolve isso em duas fases:

1. **Emissão:** o salto é emitido com rótulo provisório `"???"` e registrado em uma *lista de saltos pendentes* (`patch_list_t`).

2. **Preenchimento:** quando o rótulo de destino é conhecido, percorre-se a lista e substitui "???" pelo rótulo real.

3.1 Funções fornecidas em backpatch.h

```
/* Cria lista com uma instrução pendente */
patch_list_t *patch_list_make(tac_instr_t *instr);

/* Concatena duas listas */
patch_list_t *patch_list_merge(patch_list_t *l1, patch_list_t *l2);

/* Preenche todos os rótulos pendentes com 'label' */
void patch_list_backpatch(patch_list_t *list, const char *label);

/* Libera a lista (não as instruções TAC) */
void patch_list_free(patch_list_t *list);
```

3.2 Resultado de tradução booleana

A função `codegen_bool_expr()` retorna um par de listas:

```
typedef struct {
    patch_list_t *true_list;   /* saltos a preencher quando expr = verdadeira */
    patch_list_t *false_list;  /* saltos a preencher quando expr = falsa      */
} bool_result_t;
```

4. Infraestrutura de Back-Patching (TODOs E3-E e E3-F)

Antes de implementar a geração de TAC, os alunos devem completar as duas funções centrais do módulo `backpatch.c`. As outras duas (`patch_list_make` e `patch_list_free`) são fornecidas como exemplo.

TODO-E3-E: `patch_list_backpatch(list, label)`

Esta é a **operação que dá nome à técnica**: percorre a lista de saltos pendentes e preenche o campo `result` de cada instrução TAC com o rótulo real. A instrução TAC já foi emitida e está inserida na lista encadeada de código — você está “voltando atrás” para completá-la.

```
/* Para cada nó p da lista: */
free(p->instr->result);
p->instr->result = strdup(label);
```

Sem esta implementação, todos os saltos condicionais conservam o rótulo provisório "???" e o assembly gerado pelo `asmgen` é inválido.

TODO-E3-F: `patch_list_merge(l1, l2)`

Une duas listas ligadas de saltos pendentes em uma única lista. Necessário para combinar as listas de `&&` e `||` (por exemplo, a `false_list` de `e1 && e2` é a união das `false_lists` de `e1` e de `e2`).

```
/* Percorre l1 até o último nó, aponta seu next para l2, retorna l1. */  
/* Trate os casos l1 == NULL e l2 == NULL. */
```

Sem esta implementação, as expressões booleanas compostas perdem metade dos seus saltos e o controle de fluxo fica incorreto.

5. Algoritmos de Tradução TAC (TODOs E3-A a E3-D)

5.1 TODO-E3-A: Expressões booleanas com curto-circuito

e1 && e2 (curto-circuito: se e1 for falso, e2 não é avaliada):

```
1. r1 = codegen_bool_expr(e1)  
2. lmid = tac_new_label()  
3. patch_list_backpatch(r1.true_list, lmid) /* se e1 verdadeiro, avalia e2 */  
4. emit TAC_LABEL lmid  
5. r2 = codegen_bool_expr(e2)  
6. res.true_list = r2.true_list  
   res.false_list = merge(r1.false_list, r2.false_list)
```

e1 || e2 (curto-circuito: se e1 for verdadeiro, e2 não é avaliada):

```
1. r1 = codegen_bool_expr(e1)  
2. lmid = tac_new_label()  
3. patch_list_backpatch(r1.false_list, lmid) /* se e1 falso, avalia e2 */  
4. emit TAC_LABEL lmid  
5. r2 = codegen_bool_expr(e2)  
6. res.true_list = merge(r1.true_list, r2.true_list)  
   res.false_list = r2.false_list
```

Expressão simples (relacional, variável): já implementado no esqueleto — emite TAC_JUMPT e TAC_JUMPF e cria as listas correspondentes.

5.2 TODO-E3-B: Comando if/else

children[0] = condição, children[1] = bloco then, children[2] = bloco else (pode ser NULL).

```
cond = codegen_bool_expr(children[0])  
backpatch(cond.true_list, ltrue); emit LABEL ltrue  
[código de children[1] - then]  
if children[2] != NULL:  
    emit JUMP lend  
    backpatch(cond.false_list, lfalse); emit LABEL lfalse  
    [código de children[2] - else]  
    emit LABEL lend  
else:  
    backpatch(cond.false_list, lfalse); emit LABEL lfalse
```

5.3 TODO-E3-C: Comando while...do

```
children[0] = condição, children[1] = corpo.  
  
emit LABEL lbegin  
cond = codegen_bool_expr(children[0])  
backpatch(cond.true_list, lbody);    emit LABEL lbody  
    [código de children[1] - corpo]  
emit JUMP lbegin  
backpatch(cond.false_list, lend);    emit LABEL lend
```

5.4 TODO-E3-D: Comando for

```
children[0] = init, children[1] = condição, children[2] = step, children[3] = corpo.  
  
[código de children[0] - init]  
emit LABEL lbegin  
cond = codegen_bool_expr(children[1])  
backpatch(cond.true_list, lbody);    emit LABEL lbody  
    [código de children[3] - corpo]  
    [código de children[2] - step]  
emit JUMP lbegin  
backpatch(cond.false_list, lend);    emit LABEL lend
```

6. Geração de Assembly (Fornecida)

O módulo `asmgen.c` traduz automaticamente o TAC para assembly x86-64 (AT&T) seguindo a ABI System V. Os alunos **não precisam modificá-lo**.

6.1 Convenções de chamada (System V x86-64)

Elemento	Convenção
Argumentos inteiros	%rdi, %rsi, %rdx, %rcx, %r8, %r9
Valor de retorno	%rax
Frame pointer	%rbp
Stack pointer	%rsp (alinhado a 16 bytes antes de <code>call</code>)

6.2 Prólogo/epílogo de função

```
fname:  
    pushq    %rbp  
    movq     %rsp, %rbp  
    subq     $N, %rsp          ; N = bytes para variáveis locais (arredondado a 16)  
    ...  
    movq     %rbp, %rsp  
    popq     %rbp  
    ret
```

6.3 Mapeamento TAC → assembly (exemplos)

Instrução TAC	Assembly gerado
t = a + b	movq a, %rax / addq b, %rax / movq %rax, t
t = a * b	movq a, %rax / movq b, %rcx / imulq %rcx, %rax
t = a < b	movq a, %rax / cmpq b, %rax / setl %al / ...
ifFalse c goto L	cmpq \$0, c / je L
goto L	jmp L
print x	movq x, %rsi / leaq _fmt_int(%rip), %rdi / call printf
return v	movq v, %rax / movq %rbp, %rsp / popq %rbp / ret

7. Como compilar e usar

```
# Compilar o compilador
cd etapa3
make

# Compilar um programa LC- para assembly (saída padrão)
./compilador < tests/valid/06_fatorial.lc > fatorial.s

# Modo de depuração: inspecionar TAC
./compilador -tac < tests/valid/06_fatorial.lc

# Montar, linkar e executar (requer Linux x86-64)
gcc fatorial.s -o fatorial -no-pie
./fatorial          # deve imprimir: 120

# Executar a suite de testes
make test
```

8. Testes

Todos os 6 testes da `tests/valid/` devem ser compilados para assembly, executados e produzir a saída esperada (arquivo `.expected`):

Arquivo	O que testa	Saída esperada
01_if_simples.lc	if sem else	5 / 0
02_if_else.lc	if com else	0
03_while.lc	while...do com acumulador	10
04_for.lc	for com produto	120
05_and_or.lc	&& e com curto-circuito	1 / 2
06_fatorial.lc	Recursão com if/return	120

Os 6 testes `tests/invalid/` devem ser rejeitados pelo compilador (código de saída `!= 0`).

9. Critérios de Avaliação

Critério	Arquivo	Peso
TODO-E3-E: <code>patch_list_backpatch()</code> correto	<code>backpatch.c</code>	15%
TODO-E3-F: <code>patch_list_merge()</code> correto	<code>backpatch.c</code>	10%
TODO-E3-A: <code>&&</code> e <code>\\ </code> com curto-circuito	<code>codegen.c</code>	25%
TODO-E3-B: <code>if/else</code> com back-patching	<code>codegen.c</code>	20%
TODO-E3-C: <code>while</code> com back-patching	<code>codegen.c</code>	15%
TODO-E3-D: <code>for</code> com back-patching	<code>codegen.c</code>	15%

A verificação é feita pela execução dos programas de teste. Programas que compilam mas produzem resultado incorreto não recebem pontuação.

Nota: os TODOs em `codegen.c` dependem dos TODOs em `backpatch.c`. Implemente E3-E e E3-F primeiro — sem eles, nenhum salto condicional funciona.

Apêndice A — Otimizações Locais (Opcional, não avaliado)

Esta seção é **completamente opcional**. Não há pontuação associada. Ela existe para alunos que queiram ir além e experimentar otimizações clássicas de compiladores sobre o TAC já gerado.

O módulo está em `src/optim.c` / `src/optim.h`. Os stubs compilam e funcionam como no-op — o compilador se comporta normalmente sem que os TODOs sejam implementados.

A.1 Blocos Básicos

Um **bloco básico (BB)** é uma sequência maximal de instruções TAC com **entrada única** (a primeira instrução) e **saída única** (a última instrução). Dentro de um BB o fluxo é estritamente linear: não há desvios internos nem alvos de desvios externos.

Identificação de líderes: a primeira instrução de cada BB é um *líder*. São líderes no TAC gerado pelo compilador LC—:

Condição	Por quê é líder
Primeira instrução após <code>TAC_BEGINFUNC</code>	Início da função
Instrução <code>TAC_LABEL</code>	Possível alvo de desvio (salto para cá)
Instrução imediatamente após <code>TAC_JUMP</code> , <code>TAC_JUMPT</code> , <code>TAC_JUMPF</code> , <code>TAC_RETURN</code>	Início de bloco alternativo ou código após desvio

Exemplo — programa `if (x > 0) print x;` em TAC:

```

BEGINFUNC main
  DECL_LOCAL x
  _t1 = x > 0
  JUMPF _L1, _t1      ← fecha BB0; BB1 começa na próxima instrução
  JUMPT _L2, _t1      ← fecharia BB, mas já foi fechado acima
_L2:                  ← TAC_LABEL → líder → BB1
  PRINT x
  JUMP _L1             ← fecha BB1
_L1:                  ← TAC_LABEL → líder → BB2
  RETURN_VOID
ENDFUNC

```

Resultado: 3 blocos básicos — BB0 (declarações + comparação + desvio), BB1 (impressão + salto), BB2 (retorno).

A.2 Grafo de Fluxo de Controle (CFG)

O **CFG** é um grafo dirigido onde cada nó é um BB e cada aresta representa uma possível transferência de controle. As arestas são determinadas pela última instrução de cada BB:

Última instrução	Sucessores
TAC_JUMP L	único: bloco que começa com TAC_LABEL L
TAC_JUMPT L / TAC_JUMPF L	dois: bloco de rótulo L + bloco seguinte (queda)
TAC_RETURN / TAC_RETURN_VOID	nenhum (saída da função)
qualquer outro	único: bloco seguinte (queda linear)

A estrutura `cfg_t` em `optim.h` armazena o array de blocos; `basic_block_t` mantém arrays de ponteiros para sucessores (`succ`) e predecessores (`pred`).

A.3 TODO-OPT-A: `cfg_build()`

Implementar em `src/optim.c`. Algoritmo em duas fases:

Fase 1 — Criar BBs:

```

new_bb_next ← true
para cada instrução t de func_begin->next até func_end (exclusive):
  se new_bb_next OU t.op == TAC_LABEL:
    fechar bloco anterior (last = prev)
    criar novo bb_t e adicionar ao CFG
    new_bb_next ← (t.op in {JUMP, JUMPT, JUMPF, RETURN, RETURN_VOID})
    prev ← t
fechar último bloco (last = prev)

```

Fase 2 — Adicionar arestas:


```

para cada bloco b (índice i):
    last = b.last
    se last.op == JUMP L:
        bb_add_succ(b, cfg_find_by_label(L))
    se last.op in {JUMPT, JUMPF} L:
        bb_add_succ(b, cfg_find_by_label(L))
        bb_add_succ(b, blocks[i+1])          ← queda
    se last.op in {RETURN, RETURN_VOID}:
        (sem sucessores)
    caso contrário:
        bb_add_succ(b, blocks[i+1])          ← queda linear

```

Para verificar o resultado:

```
./compilador -cfg < tests/valid/03_while.lc
```

A.4 TODO-OPT-B: `bb_const_fold()`

Dobragem de constantes (*constant folding*): avalia em tempo de compilação expressões aritméticas cujos dois operandos são literais inteiros.

```

para cada instrução t no BB:
    se t.op in {ADD,SUB,MUL,DIV,MOD}
        E is_int_literal(t.arg1)
        E is_int_literal(t.arg2):
            res ← avaliar t.arg1 op t.arg2
            substituir instrução por TAC_COPY result = res

```

Exemplo:

```

ANTES:   _t1 = 3 + 4           →   DEPOIS:   _t1 = 7
         _t2 = _t1 * 2         →           _t2 = _t1 * 2   (sem mudança)

```

O campo `result` da instrução é preservado; apenas `op`, `arg1` e `arg2` são modificados (`free + strdup`).

A.5 TODO-OPT-C: `bb_copy_prop()`

Propagação de cópias (*copy propagation*): substitui usos de uma variável pelo valor que lhe foi copiado, enquanto esse valor permanecer inalterado no bloco.

```

tabela ← {}
para cada instrução t no BB:
    1. para cada (d → s) em tabela:
        se t.arg1 == d: substituir por s
        se t.arg2 == d: substituir por s
    2. se t define v (t.result != NULL):
        remover de tabela todo (d,s) onde d==v ou s==v
    3. se t.op == TAC_COPY e t.result e t.arg1 não nulos:

```

adicionar (t.result → t.arg1) à tabela

Exemplo:

ANTES:		DEPOIS (com bb_const_fold antes):
_t1 = x	(cópia)	_t1 = x
_t2 = _t1 + y		_t2 = x + y (+1 substituição)
_t1 = 5		_t1 = 5 (invalida _t1→x)
_t3 = _t1 + 1		_t3 = _t1 + 1 (sem mudança)

A.6 Como usar o módulo

Compilar normalmente (sem otimização):

`make`

`./compilador < tests/valid/04_for.lc > for.s`

Com otimização:

`./compilador -O < tests/valid/04_for.lc > for_opt.s`

Inspeccionar TAC antes e depois:

`./compilador -tac < tests/valid/04_for.lc > tac_antes.txt`

`./compilador -O -tac < tests/valid/04_for.lc > tac_depois.txt`

`diff tac_antes.txt tac_depois.txt`

*# Visualizar o CFG (requer **TODO**-OPT-A implementado):*

`./compilador -cfg < tests/valid/03_while.lc`

Os testes em `make test` não usam `-O` — os TODOs podem permanecer como stubs sem afetar a avaliação.